



Workpath SDK WebService Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	WebService Sample on a device.....	4
3	WebService Sample Flow	5
4	Implement the WebService	6
5	Register the WebService.....	10
6	Using client to send a request and get a response.....	11
7	Authentication Type.....	15
8	Legal notice	17
9	Support.....	18

1 Introduction

WebService Sample provides use case scenarios how to register, access, and handle web service.

1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- Name - Monospaced type is used for code fragments, function prototypes, and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 WebService Sample on a device

WebService Sample is a sample app that shows how to:

- Register an android service to use WebService
- Handle GET/PUT/POST/DELETE requests on the service

Web Service Sample

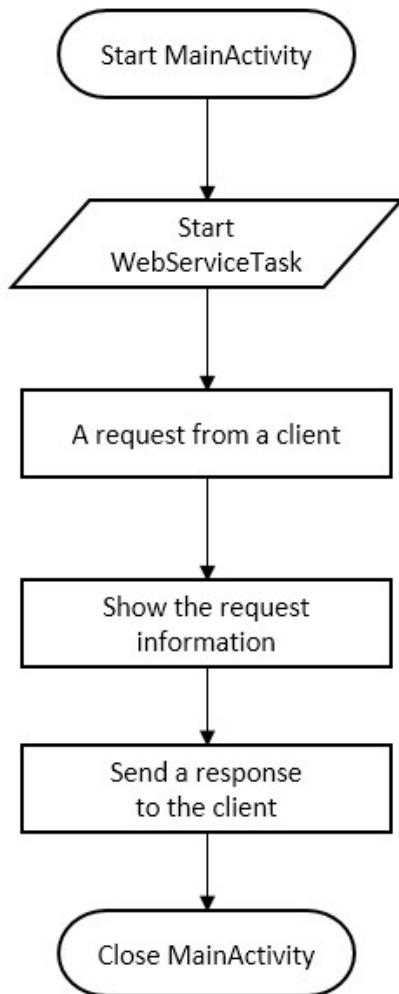
VERSION: 1.6.3 (20251111) / 17, API: 1.6.3 / 9

Request	Response
get	Body
Header {Accept-encoding=gzip, deflate, br, Hp-jetdirect-proxied-ssl=yes, Accept=*/*, Hp-proxy-session=state=Not Present#IC=Wired, Connection=keep-alive, Postman-token=317d76fa-7379-4e66-9711-67b39ddd8bf8, Host=15.26.184.176, Client-ip=192.168.3.53, path=/com.hp.workpath.sample.webservice/services/webservicetask, User-agent=PostmanRuntime/7.44.0, Content-Type=application/*, Via=1.1 15.26.184.176 (Virata-EmWeb/R6_2_1)}	
Body null	

The main activity of WebService Sample

Fields under 'Request' are information (Request method, Header, and Body) about a request of a client(From the above example, the client sends a GET request and its body contains 'WebServiceSample test' GET message.) and a field under 'Response' is an editable text field that would be returned for the client request.

3 WebService Sample Flow



Flow of WebService Sample

4 Implement the WebService

WebService needs to extend AbstractWebServices and register its Callback so that the service could handle GET/PUT/POST/DELETE methods with authenticated method. Refer to WebServiceTask.java.

1. Declare a service class by extending AbstractWebServices

```
1. public class WebServiceTask extends AbstractWebServices
```

Extend AbstractWebServices

2. Declare a Callback of AbstractWebServices and implement its methods(GET/PUT/POST/DELETE and authenticated). The methods get a HttpRequest which comes from a client as a parameter and return a HttpResponse as a result.
3. In the authenticated overridden method, we can get the value for X-Auth header via `httpRequest.getHeader(HttpRequest.HeaderKey.X_AUTH)`. Custom validation can be done here. In the sample code snippet below, X-Auth header is validated with a constant value of "123456".
 - **Warning:** WebService does not support concurrent processing so you should not run heavy work that takes too long time to be done on a method.

```
1.
2. const val X_Auth_Value="123456"
3.
4. static class WebServicesCallback implements Callback {
5.     @Override public boolean authenticated(HttpRequest httpRequest) {
6.         String msg = "Request for auth:" +
7.         httpRequest.getHeader(HttpRequest.HeaderKey.X_AUTH);
8.         Log.d(TAG, msg);
9.         setText("authenticated", httpRequest.getHeaders().toString(),
10.        httpRequest.getBody());
11.        /* Perform custom logic to validate X-AUTH header value*/
12.        if(X_Auth_Value.equals(httpRequest.getHeader(HttpRequest.HeaderKey.X_AUTH))){
13.            return true
14.        }
15.        return false
16.    }
17.
18.    @Override public HttpResponse get(HttpRequest request) {
19.        Log.d(TAG, "Get request:" + request.getAttributes().toString());
20.
21.        try {
22.            Thread.sleep(1000);
23.        } catch (Throwable throwable) {}
24.
25.        setText("get", request.getHeaders().toString(), request.getBody());
26.        Log.d(TAG, "Get request:");
27.
28.        HttpResponse response = new HttpResponse();
29.        Map<String, Object> attributes = request.getAttributes();
30.        Set<String> keys = attributes.keySet();
31.        for(String name: keys) {
32.            Log.d(TAG, name + ", " + request.getAttribute(name));
33.        }
34.        response.setStatusCode(200);
```

```

33.         response.setBody(MainActivity.getResponse());
34.         response.setStatusDescription("Error Message");
35.         return response;
36.     }
37.     @Override public HttpResponse post(HttpRequest httpRequest) {
38.         Log.d(TAG, "Post request:" + httpRequest.getAttributes().toString());
39.
40.         try {
41.             Thread.sleep(5000);
42.         } catch (Throwable throwable) {}
43.
44.         setText("post", httpRequest.getHeaders().toString(), httpRequest.getBody());
45.         Log.d(TAG, "Post request:");
46.
47.         HttpResponse response = new HttpResponse();
48.         Map<String, Object> attributes = httpRequest.getAttributes();
49.         Set<String> keys = attributes.keySet();
50.         for(String name: keys) {
51.             Log.d(TAG, name + ", " + httpRequest.getAttribute(name));
52.         }
53.
54.         response.setBody(MainActivity.getResponse());
55.         return response;
56.     }
57.     @Override public HttpResponse put(HttpRequest httpRequest) {
58.         Log.d(TAG, "Put request:" + httpRequest.getAttributes().toString());
59.
60.         try {
61.             Thread.sleep(3000);
62.         } catch (Throwable throwable) {}
63.
64.         setText("put", httpRequest.getHeaders().toString(), httpRequest.getBody());
65.         Log.d(TAG, "Put request:");
66.
67.         HttpResponse response = new HttpResponse();
68.         Map<String, Object> attributes = httpRequest.getAttributes();
69.         Set<String> keys = attributes.keySet();
70.         for(String name: keys) {
71.             Log.d(TAG, name + ", " + httpRequest.getAttribute(name));
72.         }
73.         response.setBody(MainActivity.getResponse());
74.         return response;
75.     }
76.     @Override public HttpResponse delete(HttpRequest httpRequest) {
77.         Log.d(TAG, "Delete request:" + httpRequest.getAttributes().toString());
78.         try {
79.             Thread.sleep(1000);
80.         } catch (Throwable throwable) {}
81.
82.         setText("delete", httpRequest.getHeaders().toString(), httpRequest.getBody());
83.         Log.d(TAG, "Delete request:");
84.         HttpResponse response = new HttpResponse();
85.         Map<String, Object> attributes = httpRequest.getAttributes();
86.         Set<String> keys = attributes.keySet();
87.         for(String name: keys) {
88.             Log.d(TAG, name + ", " + httpRequest.getAttribute(name));
89.         }
90.         response.setBody(MainActivity.getResponse());
91.         return response;
92.     }
93. }

```

Implemented Callback of AbstractWebServices

4. Register the callback to be called on the callback variable of AbstractWebServices.

```
1. @Override protected void onStart() {
    Log.d(TAG, "onStart()");
    callback = new WebServicesCallback();
}
```

Register the implemented callback

5. How to Configuration the Endpoint Path and Service

To configure the endpoint for your Workpath Web Service, set the following values:

- Category (Package name):
com.hp.workpath.sample.webservice
- Absolute Path (Service class name):
com.hp.workpath.sample.webservice.services.WebServiceTask
(This should be the fully qualified class name, including the package name.)

Based on this configuration, the resulting URL will be:

```
https://{device_ip}/hp/device/webservices/ext/svcmtg/com.hp.workpath
.sample.webservice/services/webservicetask
```

- https://{device_ip}/hp/device/webservices/ext/svcmtg/
→ This is a **fixed prefix** and cannot be changed.
 - com.hp.workpath.sample.webservice
→ This is the **package name**, as specified in the Category field.
 - services/webservicetask
→ This is the **service class name**, derived from the class name in the Absolute Path.
(Note: The class name is converted to lowercase in the URL)
- #### 6. Supporting subdirectories in request paths
- Previously, the WebService routing mechanism required a one-to-one mapping between a specific directory path and a callback class. If a client requested a path that included a subdirectory (e.g., /hp/device/webservices/ext/svcmtg/com.hp.workpath.sample.webservice/services/webservicetask/sub1/sub2), and no explicit mapping existed for that subdirectory, the framework would return a 404 Not Found error.
 - With the updated behavior, the framework now supports subdirectory traversal. This means:
 - A single callback class mapped to a parent directory (e.g., /hp/device/webservices/ext/svcmtg/com.hp.workpath.sample.webservice/services/webservicetask) can now handle requests to any subdirectory under it.
 - The full request path is passed to the callback via the :path key in the request attributes, and this value represents the logical path relative to the registered service root. For instance, even if the actual request URL is /hp/device/webservices/ext/svcmtg/com.hp.workpath.sample.webservice/web

servicetask/sub1/sub2, the value provided in the :path attribute will be com.hp.workpath.sample.webservice/services/webservicetask/sub1/sub2.

- However, note that the number of subdirectory levels supported is limited to 10, and requests exceeding this depth may not be routed correctly.
- This allows developers to dynamically resolve and serve content from nested paths without needing to register separate callback classes for each subdirectory.
- Sample code snippet

```
1. Map<String, String> requestHeaders = request.getHeaders();
2.
3. // Extract the full path from the request headers
4. String fullPath = requestHeaders.get(":path");
5.
6. // Remove the prefix to get the subPath
7. String subPath =
   fullPath.replaceFirst("com.hp.workpath.sample.webservice/webservicetask/", "");
8.
9. File file = new File(BASE_DIRECTORY, subPath);
10.
11. if (file.exists()) {
12.     // Serve the file or process accordingly
13. } else {
14.     // Return 404 or appropriate error
15. }
```

5 Register the WebService

An application requires registering an android service on AndroidManifest.xml to handle GET/PUT/POST/DELETE methods with authenticated method from a client.

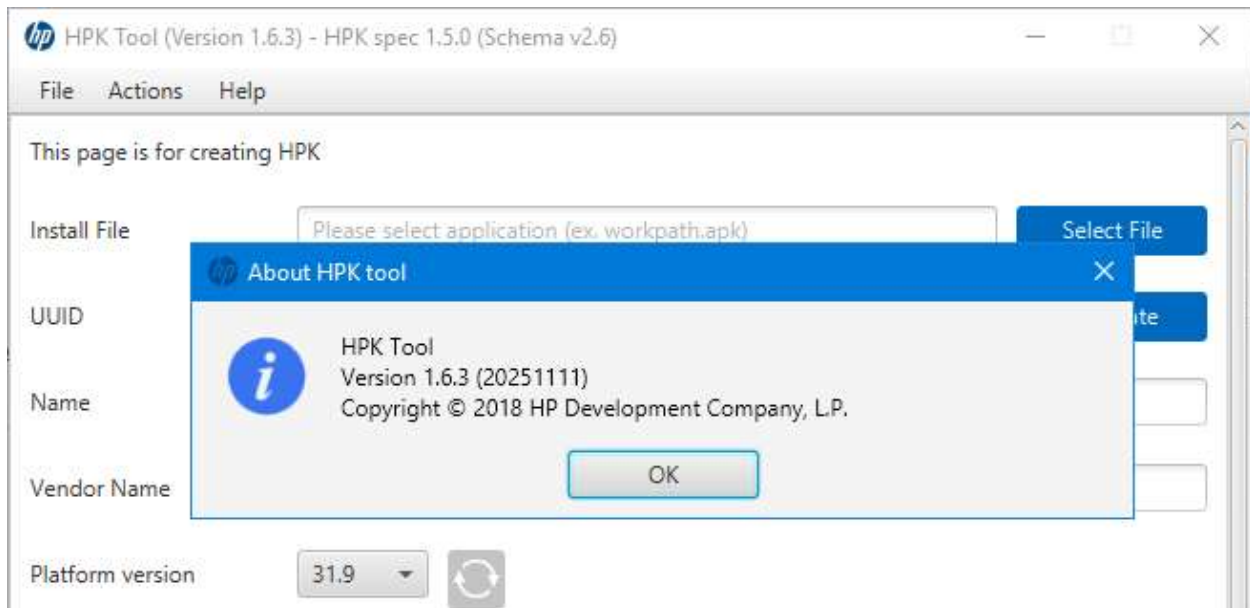
The sample shows how to register the service on AndroidManifest.xml.

1. Place the service on AndroidManifest.xml.

```
1. <service android:name=".services.WebServiceTask">
2.   <intent-filter>
3.     <action android:name="com.hp.workpath.api.action.WEBSERVICES"/>
4.   </intent-filter>
5. </service>
```

Register the service on AndroidManifest.xml

Also, while making an .hpk file, the methods and the service should be registered so that a client application could access the methods. Please refer to the HP WorkpathSDK-HPKTool-UserGuide document. HPK Tool version should be at least V1.6.0 as the figure.



HPK Tool > Help > About > HPK Tool Version

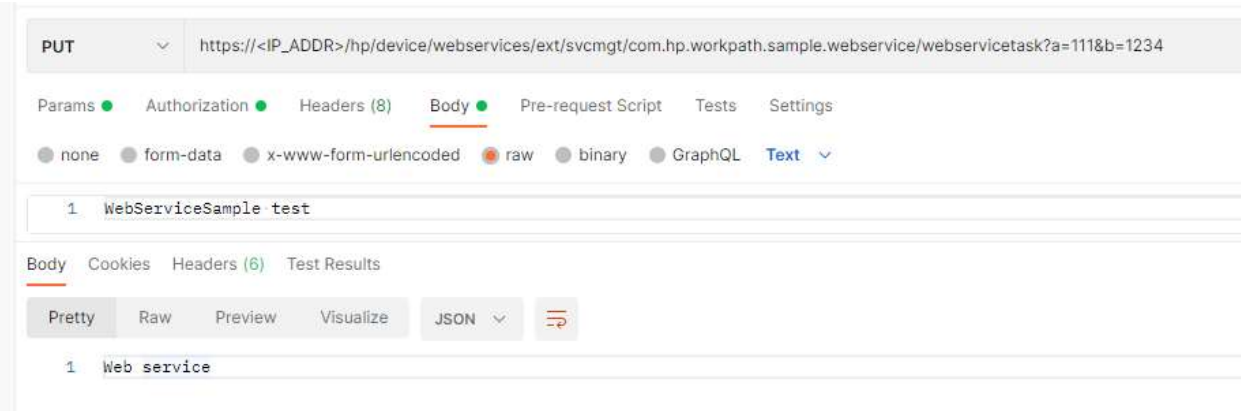
6 Using client to send a request and get a response

The below picture shows the status before a client makes a request. The client will get a 'Web service' message because the message is written on the Response body editable text field.

Web Service Sample		VERSION: 1.6.3 (20251111) / 17, API: 1.6.3 / 9
Request		Response
Header		Body
Body		

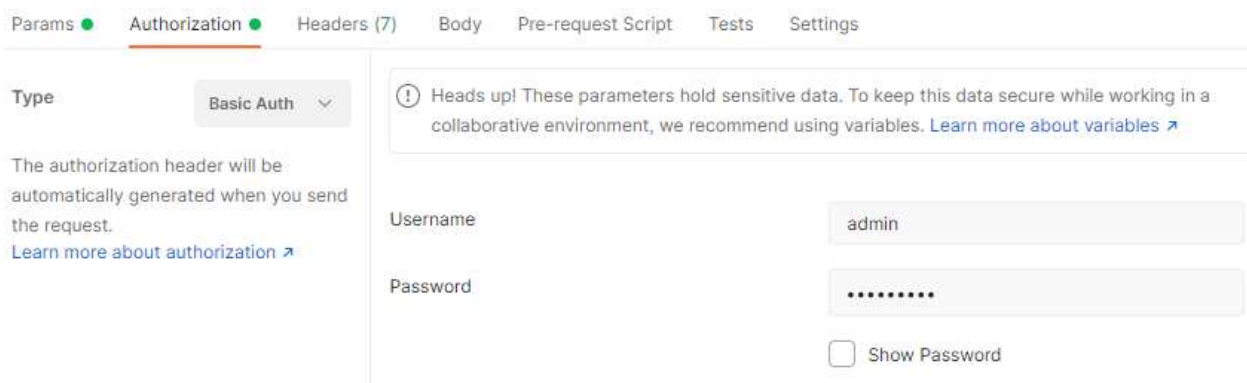
Run the WebServiceSample application

If the client sends a request as the below picture – PUT method, 'WebServiceSample test' on a body – then the client gets 'Web service' message that was set from the WebServiceSample application. (You can use any client to make this request)



Client screen

Http Authorization Basic(RFC 7617 base64-encoded credentials) should be set like below if there is exist EWS admin password.



If your credential is not allowed, you will get the response 401 error code.



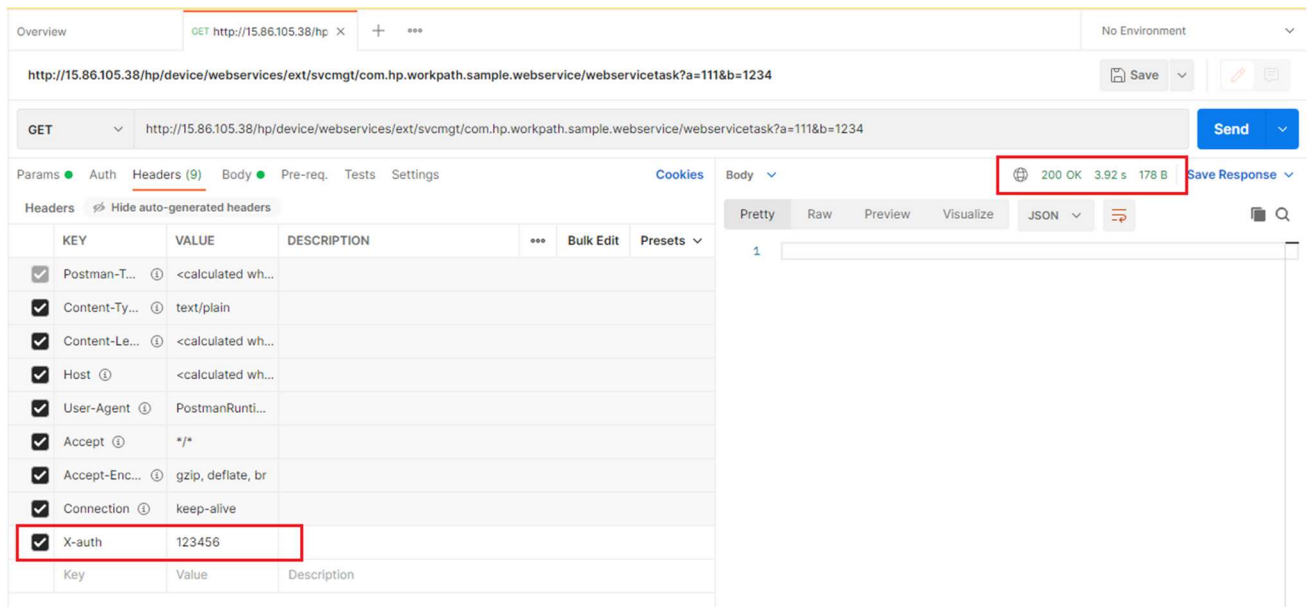
In this case, WebServiceSample application cannot receive any requests.

And WebServiceSample application will show the request information as the below picture. Fields under 'Request' will be filled.

Web Service Sample		VERSION: 1.6.3(20251111) / 17, API: 1.6.3 / 9	
Request		Response	
get		Body	
Header {Accept-encoding=gzip, deflate, br, Hp-jetdirect-proxyed-ssl=yes, Accept=*/*, Connection=keep-alive, Host=15.26.184.176, User-agent=PostmanRuntime/7.44.0, Via=1.1 15.26.184.176 (Virta-EmWeb/R6_2.1, Hp-proxy-session=state=Not Present/NIC=Wired, Postman-token=0b681dde-92fc-4ec0-b7f7-beb07a6ae9b9, Client-ip=192.168.3.53, .path=/com.hp.workpath.sample.webservice/services/webservicetask, Content-length=21, Content-Type=application/*)}			
Body WebServiceSample test			

After getting the request from the client

To use GET/PUT/POST/DELETE methods with the authenticated method, put X-Auth field on a request header and handle a value of the X-Auth field on the authenticated method of the service. If the request is authenticated from the authenticated method of the service's callback then return true to handle the HTTP request otherwise, return false to ignore the request. The below is an example to set X-Auth field from the postman.



Put X-Auth field on the header from the postman

7 Authentication Type

Webservice supports three types of authentication types which are:

- None
 - Pass the call to the app (no authentication)
- Admin
 - Framework will validate the Authorization credential (EWS Admin)
 - If valid, pass the call to the app without an Authorization header.
 - If not valid, Framework returns 401 immediately (no call is made to the app)
- X-Auth
 - Pass the call to the app with the custom X-Auth header
 - App will validate the X-Auth credential (and respond as appropriate)

These authentication types needs to be configured at the time of HPK creation via HPK Tool.

Assume for our Webservice anyone can GET, a custom X-Auth header is needed to PUT, but admin is required to POST and DELETE. On this case, the hpk.xml could look like this:

```
{  "method": "get",    "category": "com.hp.workpath.sample.webservice",
  "absolutePath": "com.hp.workpath.sample.webservice.services.WebServiceTask",
  "authtype": "none" },
```

```
{  "method": "put",    "category": "com.hp.workpath.sample.webservice",
  "absolutePath": "com.hp.workpath.sample.webservice.services.WebServiceTask",
  "authtype": "x-auth" },
```

```
{  "method": "post",   "category": "com.hp.workpath.sample.webservice",
  "absolutePath": "com.hp.workpath.sample.webservice.services.WebServiceTask",
  "authtype": "admin" },
```

```
{  "method": "delete", "category": "com.hp.workpath.sample.webservice",
  "absolutePath": "com.hp.workpath.sample.webservice.services.WebServiceTask",
  "authtype": "admin" }
```

 WebService ✕

Method

☒ Get ☐ Put ☐ Post ☐ Delete

Category

Absolute Path

Auth Type

☒ None ☐ X-Auth ☐ Admin

Add

Cancel

This would allow the Workpath framework to apply the necessary checks before forwarding the request to the application.

8 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

9 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.